


Dynamic Programming

i/p $\rightarrow$ coins $\rightarrow [1, 2, 5]$ $\rightarrow$ infinite supply
amount $\rightarrow [11]$
minimum no of coins $\rightarrow$ o/p $\rightarrow 3$ $[5+5+1]$

coin $[2]$

target $\rightarrow 3$ ans $\rightarrow [-1]$

[1]

$T \rightarrow \underline{\underline{0}} \rightarrow \text{o/p} \rightarrow \underline{\underline{0 \text{ coins}}}$

[1, 2, 3] $x \rightarrow 7$

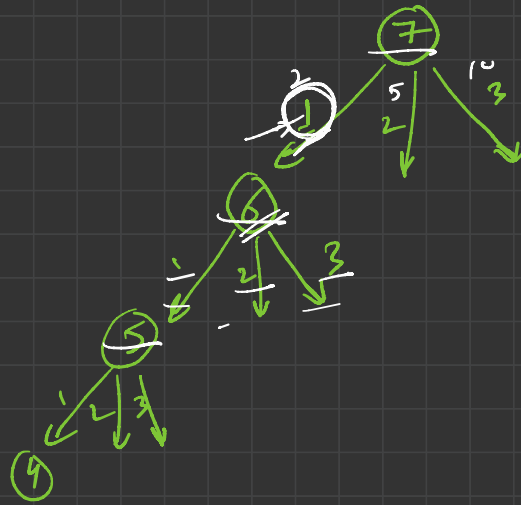
1+1+1+1+1+1 $\rightarrow \textcircled{7}$

2+2+2+1 $\rightarrow \textcircled{4}$

$\textcircled{3+3+1} \rightarrow \underline{\underline{3}}$

$\rightarrow \text{Reci} + \text{Mem} \rightarrow \text{tabulation} \rightarrow \underline{\underline{S.O}}$

① Recursion



memoization

[1, 2, 3]

7

[2, 5, 10]

// B.C

if (target == 0) {
 return 0;

if (target < 0)
 return INT_MAX;

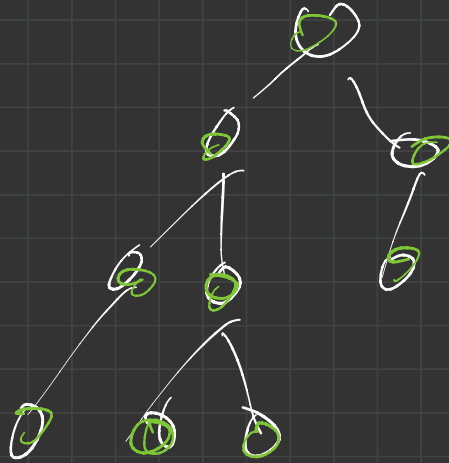
int min = INT_MAX;

for (int i = 0; i < coins.size(); i++)
 min = min(1 + f(target - coins[i]), min);

if ($ans \neq \infty$)

3 $mini = \min (mini, \lceil \frac{1}{tau} \rceil)$

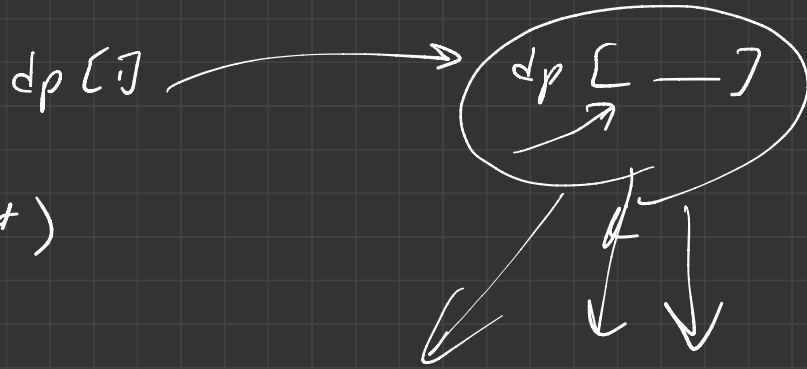
return ans



dp[i]

n

→ k → min no of coins
↓ ↓ ↓



for (1 $\rightarrow$ $\leq$ amt)

$dp[1]$

$dp[2]$

$dp[3]$

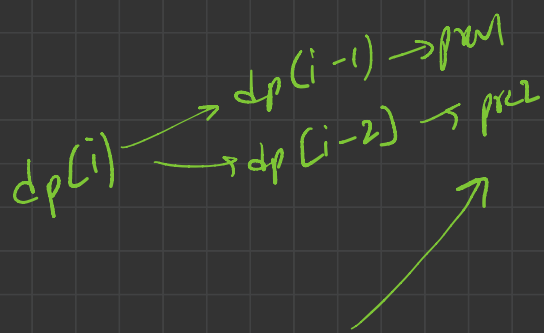
$dp[amount]$

① Recursion: - T.C $\rightarrow$ exp
+ Mem

T.C $\rightarrow$ $x * n$
amount no. of coins

S.C $\rightarrow$ $O(x)$

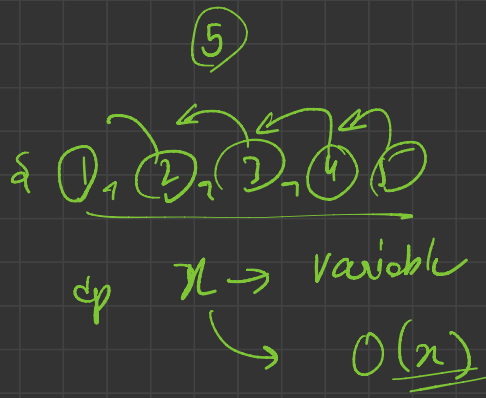
② Tabulation $\rightarrow$ T.C $\rightarrow$ Mem }



Space Optimisation
↓
?

S.C. $\rightarrow$ $O(n)$
↓
 $O(1)$

$dp[i] \rightarrow dp[i - \text{num}[j]];$



$i-1$

$i-2$

$i-3$

$i-4$

$i-5$

,

,

0

